ITA06 – MACHINE LEARNING

ASSIGNMENT 4 – CO4

INSTANCE-BASED LEARNING, k-NEAREST NEIGHBOUR, LOCALLY WEIGHTED REGRESSION, RBF NETWORKS AND CASE-BASED LEARNING

Student Name: A. Lakshmi Devi

Register Number: 192525176

Programme / Branch: AI & ML (Artificial Intelligence & Machine Learning)

Course Code: ITA0611

Course Name: Machine Learning

Assignment: 4 – CO4

Total Marks: 50

Duration: 1 Week

CONTENTS

1. Q1 – k-Nearest Neighbour classification
2. Q2 – Distance weighting, feature scaling and dimensionality
3. Q3 – Locally weighted regression
4. Q4 – Radial basis function network
5. Q5 – Case-based reasoning
6. Q6 – Lazy versus eager learning
7. Q7 – Day-4 redesign challenge
8. Conclusion
9. Appendix – Python implementation guide

INTRODUCTION

This report follows the supplied ITA06 Machine Learning Assignment 4 – CO4 brief (50 marks). It covers instance-based learning, k-nearest neighbours, feature scaling, locally weighted regression, radial basis function networks, case-based reasoning and deployment trade-offs.

Evidence note: the assignment requires reproducible synthetic-data experiments, timing measurements, plots and test-set results. The manual calculations are provided below. Experimental results that depend on execution are not fabricated; the appendix provides a reproducible implementation framework and the report identifies where actual outputs must be inserted. The supplied brief (pages 6–7) requires source code, data-generation details, seeds, partitions and raw results.

Q1 – APPLY k-NEAREST NEIGHBOUR LEARNING TO A CLASSIFICATION PROBLEM (6 MARKS)

1.1 Why k-NN is a lazy learner

k-NN stores the labelled training instances and delays generalisation until a query arrives. Its training phase mainly consists of storing records and preparing any required scaling or index. At prediction time it computes distances to stored examples, selects the k nearest neighbours and votes. Therefore training is fast, but prediction and memory costs can be high.

1.2 Euclidean distance table for query (3,3)

Euclidean distance: $d(x,q)=\sqrt{[(x1-q1)^2+(x2-q2)^2]}$. The query is q=(3.0,3.0).

Point	Coordinates	Euclidean distance	Class	Rank
P4	(3, 2.5)	0.5	Worn	1
P3	(2, 3)	1.0	Normal	2
P5	(4, 4)	1.4142	Worn	3
P6	(4.5, 3)	1.5	Worn	4
P7	(3.5, 1)	2.0616	Normal	5
P1	(1, 2)	2.2361	Normal	6
P2	(2, 1)	2.2361	Normal	7
P8	(5, 5)	2.8284	Worn	8

For k=1, nearest point is P4 (distance 0.5000, Worn), so prediction = Worn.

For k=3, neighbours are P4 (Worn), P3 (Normal) and P6 (Worn). Vote: Worn=2, Normal=1. Prediction = Worn.

For k=5, neighbours are P4 (Worn), P3 (Normal), P6 (Worn), P5 (Worn), P1 (Normal). Vote: Worn=3, Normal=2. Prediction = Worn.

1.3 Manhattan distance for k=3

Manhattan distance: $d_1(x,q)=|x1-q1|+|x2-q2|$.

Rank	Point	Manhattan distance	Class
1	P4	0.5	Worn
2	P3	1	Normal
3	P6	1.5	Worn
4	P5	2	Worn
5	P7	2.5	Normal
6	P1	3	Normal
7	P2	3	Normal
8	P8	4	Worn

The three nearest are P3 (Normal, 1.0), P4 (Worn, 0.5), and P6 (Worn, 1.5). Therefore the majority vote is Worn (2 versus 1). Note that P3 and P4 tie at distance 1.0? The listed ranking is ordered by distance; P4 is first at 0.5, P3 and P6 are 1.0 and 1.5 respectively.

1.4 Query where prediction changes with k

Choose q_2=(2.5,1.5). Its closest records are P2=(2,1), Normal, distance 0.707; P7=(3.5,1), Normal, distance 1.118; P4=(3,2.5), Worn, distance 1.118; P1=(1,2), Normal, distance 1.581; P3=(2,3), Normal, distance 1.581. For k=1 prediction is Normal; for k=3 the three nearest contain two Normal and one Worn, so Normal; for k=5 all five contain four Normal and one Worn, so still Normal. A changing example can be found at q=(3.4,2.2): nearest P4 Worn; among k=3, P4 Worn, P3 Normal, P6 Worn → Worn; among k=5, P4

Worn, P3 Normal, P6 Worn, P5 Worn, P2 Normal → Worn. In this small dataset, a useful changing query is q=(2.7,2.0): k=1 P4 Worn; k=3 P4 Worn, P3 Normal, P1 Normal → Normal; k=5 adds P2 Normal and P7 Normal → Normal.

This demonstrates that the local vote can change as k changes. Small k produces a flexible, high-variance boundary; large k smooths the boundary and can create high bias. The chosen k should be selected using validation data, not the test set.

1.5 1-NN Voronoi regions and implementation

For 1-NN, the plane is divided into Voronoi cells: each cell contains points closer to one training sample than to any other. The class boundary consists of portions of perpendicular bisectors between neighbouring points with different labels. The diagram can be drawn by plotting the eight coordinates, colouring each grid location by its nearest training point, and overlaying the sample markers.

Tie-breaking rule: sort equal distances by the point ID in ascending order. For class-vote ties, choose the class of the closest neighbour; if still tied, use alphabetical order.

```
import numpy as np
from collections import Counter
X=np.array([[1,2],[2,1],[2,3],[3,2.5],[4,4],[4.5,3],[3.5,1],[5,5]],float)
y=np.array(["Normal","Normal","Normal","Worn","Worn","Worn","Normal","Worn"])
def knn_predict(q,k,metric="euclidean"):
    if metric=="euclidean": d=np.sqrt(((X-q)**2).sum(axis=1))
    else: d=np.abs(X-q).sum(axis=1)
    order=sorted(range(len(X)),key=lambda i:(d[i],i))
    ids=order[:k]; counts=Counter(y[ids])
    best=max(counts.values())
    tied=[c for c,n in counts.items() if n==best]
    if len(tied)==1: return tied[0]
    return y[ids[0]]
print(knn_predict(np.array([3.,3.]),3))
# Verify with sklearn:
from sklearn.neighbors import KNeighborsClassifier
model=KNeighborsClassifier(n_neighbors=3,metric="euclidean").fit(X,y)
print(model.predict([[3.,3.]]))
```

Q2 – INVESTIGATE DISTANCE WEIGHTING, FEATURE SCALING AND THE CURSE OF DIMENSIONALITY (6 MARKS)

Prediction before running: standardisation should improve k-NN when features have different units or ranges because otherwise the large-scale feature dominates distance. Distance weighting $w=1/d^2$ should give nearby examples more influence. Adding irrelevant features should make distances less discriminative and usually reduce accuracy.

2.1 Reproducible experiment design

Generate 1,000 observations with two informative features: x1 drawn on a large scale (e.g. normal mean 100, SD 25) and x2 on a small scale (e.g. uniform 0–1). Define a nonlinear binary target from their standardised latent values, add controlled noise, and use a fixed seed. Split into train/validation/test once (for example 60/20/20) with stratification. Fit StandardScaler only on training data.

2.2 k and distance-weighting comparison

Evaluate $k \in \{1,3,5,7,9,15,25\}$ on validation accuracy for unscaled and scaled features. Select the best k separately for each preprocessing option. At the selected k compare uniform voting with inverse-square weighting $w_i=1/(d_i^2+\varepsilon)$, where ε prevents division by zero. Repeat at least three seeds and report mean and standard deviation (or range).

Experiment	Expected observation	Reason
Unscaled vs scaled	Scaling should help when both features are informative but on very different ranges.	Euclidean distance is scale-sensitive.
Uniform vs $1/d^2$	Weighted vote can improve local decisions when nearest points are more relevant.	Closer neighbours receive greater weight.
Add irrelevant features	Accuracy generally degrades; distance ratios tend toward 1.	Noise dimensions add distance without class information.

2.3 Curse of dimensionality and test discipline

In high dimensions, pairwise distances become increasingly similar: the nearest and farthest neighbour distances become closer in relative terms. Nearest-neighbour identity then carries less information. Remedies include feature selection, feature weighting and dimensionality reduction (e.g. PCA fitted on training data only). The test set must not be used to choose k because that leaks information and makes the final performance estimate optimistically biased.

For each irrelevant-feature count {0,2,5,10,20,50}, record validation accuracy and ratio d_nearest/d_farthest (or clearly define the requested nearest-to-farthest ratio). Plot both against feature count. Use the validation set for selection and evaluate the chosen configuration once on the held-out test set.

Q3 – DEVELOP AND VERIFY LOCALLY WEIGHTED REGRESSION (7 MARKS)

3.1 Model and weighted least-squares derivation

For query xq, locally weighted linear regression fits θ specifically for that query. With design matrix X whose rows are [1,xi], diagonal weight matrix W with $w_i=\exp[-(x_i-x_q)^2/(2\tau^2)]$, minimise $J(\theta)=(y-X\theta)^T W(y-X\theta)$. Differentiating and setting to zero: $-2X^TW(y-X\theta)=0$, so $X^TWX\theta=X^TWy$. Therefore $\theta=(X^TWX)^{-1}X^TWy$, when the matrix is invertible.

3.2 Manual prediction at xq=3.5, τ=1

Data: x=(1,2,3,4,5), y=(1.2,1.9,3.2,3.9,5.5). Gaussian weights are:

xi	yi	wi
1	1.2	0.043937
2	1.9	0.324652
3	3.2	0.882497
4	3.9	0.882497
5	5.5	0.324652

Using X rows [1,xi], the weighted matrices are:

X^TWX = [[2.458236, 8.493983], [8.493983, 31.521281]]

X^TWy = $[8.720881, 32.453268]^T$

θ = $[\text{intercept, slope}]^T$ = $[-0.143129, 1.068136]^T$. Therefore $\hat{y}(3.5) = 3.595346$.

Ordinary global least-squares line has θ = $[-0.040000, 1.060000]^T$ and predicts 3.670000 at x=3.5.

3.3 Nonlinear experiment and bandwidth behaviour

Generate 100 samples from y=sin(x)+Gaussian noise using a fixed seed; create fixed train/validation partitions. Fit locally weighted regression for at least four τ values, such as 0.05, 0.2, 0.7 and 5.0. Plot each fitted curve and report train/validation mean squared error. Compare against global linear and polynomial regression; measure time to predict 100 query points.

As $\tau\to 0$, only extremely close observations receive meaningful weight; predictions become highly local and unstable in sparse regions. As $\tau\to\infty$, weights become nearly equal and the fit approaches ordinary global linear regression. Small τ tends toward low bias/high variance; large τ tends toward higher bias/lower variance. Predictions are least reliable near the range ends and in sparse regions because fewer nearby observations constrain the fit.

Unlike a global model, locally weighted regression has no single fixed parameter vector: it solves a weighted regression for each query. This increases prediction-time computation.

Q4 – DESIGN AND EVALUATE A RADIAL BASIS FUNCTION NETWORK (7 MARKS)

4.1 RBF model

An RBF network has Gaussian hidden units $\varphi_j(x)=\exp[-\|x-c_j\|^2/(2\sigma^2)]$ and a linear output $f(x)=b+\Sigma_j w_j\varphi_j(x)$. Centres c_j determine where each unit responds; σ controls the spatial width; output weights combine the hidden responses. Centres may be selected by k-means on training data, while the output layer is fitted by least squares or logistic regression.

4.2 XOR hidden-feature calculation

Given $c_1=(0,0)$, $c_2=(1,1)$, $\sigma^2=0.5$, the denominator $2\sigma^2=1$, so $\varphi_j(x)=\exp(-\text{squared distance})$.

Input	XOR target	φ1 (centre 0,0)	φ2 (centre 1,1)
(0,0)	0	1.000000	0.135335
(0,1)	1	0.367879	0.367879
(1,0)	1	0.367879	0.367879
(1,1)	0	0.135335	1.000000

For the two positive XOR points, (0,1) and (1,0), the feature vector is $(e^{-1},e^{-1})\approx(0.3679,0.3679)$. For the negative points, (0,0) maps to $(1,e^{-2})\approx(1,0.1353)$, and (1,1) maps to $(e^{-2},1)\approx(0.1353,1)$. A separating line is $\varphi1+\varphi2=0.9$: positive points sum≈0.7358 (below 0.9), negatives sum≈1.1353 (above 0.9). Thus predict XOR=1 when $\varphi1+\varphi2<0.9$.

Least-squares output for labels [0,1,1,0] using a linear output with bias is obtained by solving $\Phi_aug\ w\approx y$, where Φ_aug has a leading column of ones and two RBF columns. Since the two positive feature rows are identical and the two negative rows are symmetric, the least-squares solution is approximately bias=1.0, w1=−1.0, w2=−1.0, giving scores: (0,0)≈−0.1353, (0,1)≈0.2642, (1,0)≈0.2642, (1,1)≈−0.1353. The threshold 0.5 does not classify positives correctly; instead a decision threshold around 0.1 separates them. A more direct separating score is $f=0.9-\varphi1-\varphi2$, positive for XOR positives and negative for XOR negatives.

4.3 RBF experiment and comparison

Generate a fixed-seed two-class crescent/two-moons dataset with 1,000 observations and fixed partitions. Fit k-means centres using training data only. Try centre counts {2,5,10,20,50} and several σ values (e.g. 0.2,0.5,1.0,2.0 after scaling). Fit output weights by least squares or logistic regression. Record train/validation accuracy and parameter count (M centres in d dimensions: Md centre coordinates + M widths if learned + M output weights + bias). Plot underfit, well-fit and overfit boundaries. Compare the best RBF model with the best Q2 k-NN model on the same test partition.

RBF networks replace many stored examples with a smaller set of centres and can make prediction faster. k-NN stores all examples and adapts locally without fitting a global output layer. MLPs learn hidden representations by gradient descent; RBF centres and widths are often selected separately (e.g. k-means and a width heuristic) and only output weights are solved directly. RBF performance can be sensitive to σ and centre count.

Q5 – BUILD AND ASSESS A CASE-BASED REASONING SYSTEM (5 MARKS)

5.1 CBR cycle and similarity

Case-based reasoning follows four stages: Retrieve the most similar prior case; Reuse its solution; Revise the solution using current constraints or expert feedback; Retain the validated new case for future use. Unlike ordinary k-NN on numeric vectors, CBR stores structured cases with problem descriptions, solutions and possible revision history.

Use exact-match similarity for Noise and Overheating (1 for match, 0 otherwise). For ordered Pressure, map Low=0, Normal=1, High=2 and define simP=1−|rank difference|/2. Choose weights Noise=0.30, Overheating=0.40 and Pressure=0.30, giving more importance to overheating because it is directly related to the maintenance problem. Overall similarity is weighted sum.

5.2 Retrieve for new case (Yes, Yes, Low)

Case	Noise sim	Overheat sim	Pressure sim	Weighted similarity	Stored solution
C1	1	0	1	0.600	Replace seal
C2	0	1	0.5	0.550	Clean cooling fins
C3	1	1	0	0.700	Reduce load and inspect bearing
C4	0	0	1	0.300	Refill lubricant
C5	1	0	0.5	0.450	Tighten mounting
C6	0	1	0	0.400	Replace coolant pump

The highest similarity is C3 (0.700), so the initial proposed solution is “Reduce load and inspect bearing.” An engineer may revise this after inspection—for example, if bearing inspection is normal but coolant flow is blocked, revise to “Clean cooling fins and check coolant circulation.” Retain the validated case with its confirmed outcome, date, and diagnostic evidence.

If the Overheating weight is doubled from 0.40 to 0.80 and weights are renormalised to Noise=0.2143, Overheating=0.5714, Pressure=0.2143, C3 remains the strongest match (0.7857); C2 becomes 0.6786 and C6 becomes 0.5714. This increases the priority of cases with matching overheating status.

5.3 Case-base growth

As the case base grows, brute-force retrieval cost grows approximately linearly with the number of cases. Quality may improve because more precedents are available, but redundant, obsolete or conflicting cases can reduce reliability. Maintenance policies include duplicate detection, ageing/forgetting obsolete cases, retaining validated outcomes, resolving conflicts by recency or expert confidence, and indexing cases for faster retrieval.

```
cases = [
 ("C1","Yes","No","Low","Replace seal"),
 ("C2","No","Yes","Normal","Clean cooling fins"),
 ("C3","Yes","Yes","High","Reduce load and inspect bearing"),
 ("C4","No","No","Low","Refill lubricant"),
 ("C5","Yes","No","Normal","Tighten mounting"),
 ("C6","No","Yes","High","Replace coolant pump")]
rank={"Low":0,"Normal":1,"High":2}
def retrieve(query, weights=(.3,.4,.3)):
    qn,qo,qp=query; result=[]
    for cid,n,o,pr,solution in cases:
        sN=float(n==qn); sO=float(o==qo)
        sP=1-abs(rank[pr]-rank[qp])/2
        score=weights[0]*sN+weights[1]*sO+weights[2]*sP
        result.append((score,cid,solution))
    return sorted(result,reverse=True)
print(retrieve(("Yes","Yes","Low")))
# Retention after expert validation
def retain(new_case):
    global cases
    cases.append(new_case)
```

Q6 – COMPARE LAZY AND EAGER LEARNING UNDER COST AND NOISE CONSTRAINTS (6 MARKS)

Choose k-NN as the instance-based learner and logistic regression as the eager learner. k-NN is a natural lazy baseline from Q1/Q2; logistic regression learns a compact parameter vector during training and predicts quickly.

Measure as training size grows 200→5,000	k-NN (lazy)	Logistic regression (eager)
Training time	Low; mostly storing records.	Higher; iterative optimisation over training data.
Prediction time	Increases with stored records for brute-force search.	Nearly constant for fixed feature count.
Memory	Stores all training instances and labels.	Stores a fixed number of weights and bias.
Accuracy	Can improve with more representative local examples; sensitive to noise and scaling.	Can improve with data but limited by linear decision boundary.
Label noise	Local corrupted labels can directly affect neighbour votes.	Noise influences fitted parameters but is averaged across examples.

Run five training sizes (e.g. 200, 500, 1,000, 2,500, 5,000) with at least three seeds. Record training time, prediction time for 1,000 queries, number of stored instances/parameters and validation accuracy. Flip 0%, 10%, 20% of training labels at random using a fixed seed and compare degradation.

Compare brute-force neighbour search with a k-d tree or ball tree at low and high dimensionality. Tree indexes can accelerate low-dimensional search, but their advantage often disappears as dimensionality increases because pruning becomes ineffective. Report the observed crossover from measured timings.

Deployment recommendation: choose logistic regression when prediction latency and memory are strict and the decision boundary is reasonably linear. Choose k-NN when local nonlinear structure is important, the dataset is modest, memory is available and prediction latency is acceptable. A compact RBF network is an intermediate option when nonlinear boundaries are needed with fewer stored prototypes. Conclusions are limited by the tested dataset, seeds, sizes and noise levels.

Q7 – DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE (13 MARKS)

Selected limitation: high prediction cost and sensitivity to irrelevant dimensions in k-NN

A. Diagnose the bottleneck (2 marks)

Brute-force k-NN compares each query with every stored instance. Prediction cost grows with the training-set size and feature dimension. Irrelevant dimensions also distort distances and reduce neighbour quality. This limitation comes from distance representation and computational cost.

B. Define a measurable objective (2 marks)

Baseline: standardised brute-force k-NN using all features. Redesign objective: reduce stored reference points and prediction time by at least 50% while keeping validation accuracy within 2 percentage points of the baseline. These are target criteria to test, not claimed outcomes.

C. Revised learning strategy (4 marks)

Use prototype-based RBF learning: select a smaller set of centres using k-means on training data only, remove irrelevant features using training-only feature selection, and fit a linear output layer on the RBF activations. This replaces every stored instance with a compact set of representative centres. It addresses memory and brute-force prediction cost while retaining nonlinear decision boundaries.

```
Training set → fit scaler / feature selector on training only
          → choose M representative centres using k-means
          → calculate Gaussian RBF features ϕj(x)
          → fit output weights on training labels
Query → transform using saved scaler/selected features
          → evaluate M RBF units → linear output → class
```

Additional cost: k-means training and centre selection. Deployment memory is approximately proportional to M×d rather than N×d, where M is the number of centres and N is the number of training instances. New limitations include sensitivity to centre count, initialisation and σ; if M is too small, the model underfits.

D. Predict before implementing (1 mark)

Prediction: the RBF prototype model will use fewer stored vectors and reduce prediction time compared with brute-force k-NN. Accuracy should remain within the 2-percentage-point tolerance if the chosen centres adequately cover the class structure.

E. Implement and validate (2 marks)

Use identical train/validation/test partitions and seeds for both methods. Compare validation accuracy, test accuracy after selection, prediction time per 1,000 queries and stored vector count. Sweep centre count and σ using validation data only. Include a timing table and decision-boundary plots. Do not invent measured values; insert outputs from the executed code.

F. Justify the final design (2 marks)

Adopt the RBF redesign only if it meets the accuracy tolerance and achieves the intended time/memory reduction. Otherwise retain k-NN or increase the number of centres. k-NN remains preferable when data are small, new examples must be incorporated immediately without retraining, and exact local exemplar retrieval is valuable.

CONCLUSION

k-NN is simple and flexible but defers computation to prediction time and is sensitive to scale, k and irrelevant dimensions. Locally weighted regression adapts a linear fit to each query, with τ controlling locality and the bias–variance trade-off. RBF networks map inputs into local Gaussian features and can represent nonlinear boundaries with a compact set of centres. Case-based reasoning reuses structured prior solutions through retrieve, reuse, revise and retain. Deployment choices must balance accuracy, latency, memory, robustness to noise and maintainability.

APPENDIX – EXPERIMENT CHECKLIST AND CODE FRAMEWORK

```
# Locally weighted regression prediction
import numpy as np
def lwr_predict(x_train, y_train, xq, tau):
    X=np.c_[np.ones(len(x_train)), x_train]
    w=np.exp(-((x_train-xq)**2)/(2*tau**2))
    W=np.diag(w)
    theta=np.linalg.pinv(X.T@W@X)@(X.T@W@y_train)
    return np.array([1.,xq])@theta

# Inverse-square weighted k-NN vote
def weighted_vote(distances, labels, eps=1e-9):
    weights=1.0/(np.asarray(distances)**2+eps)
    scores={}
    for w,label in zip(weights,labels):
        scores[label]=scores.get(label,0.0)+w
    return max(scores,key=scores.get)
```

```
# RBF feature matrix
def rbf_features(X, centers, sigma):
    sq=((X[:,None,:]-centers[None,:,:])**2).sum(axis=2)
    return np.exp(-sq/(2*sigma**2))

# Suggested fixed-seed two-moons experiment and partitions
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
X,y=make_moons(n_samples=1000,noise=.20,random_state=42)
X_train,X_temp,y_train,y_temp=train_test_split(
    X,y,test_size=.40,random_state=42,stratify=y)
X_val,X_test,y_val,y_test=train_test_split(
    X_temp,y_temp,test_size=.50,random_state=42,stratify=y_temp)
# Fit scalers, feature selection, k-means centres on X_train only.
# Select k, tau, RBF centre count and sigma on X_val.
# Evaluate the selected final configuration once on X_test.
```

Before submission, attach: source code/notebook; fixed seeds and data partitions; distance tables and Voronoi plot; Q2 scaling and dimensionality plots; Q3 fitted curves and timing table; Q4 RBF hidden-feature and decision-boundary plots; Q5 retrieval ranking and retention example; Q6 timing/noise/index comparison; and Q7 baseline-versus-redesign evidence. The assignment brief also expects you to explain distance votes, LWR calculations, k/τ/centre/σ choices and data leakage during viva.